Loss Functions and the Deep Learning Intuition

Lukas Galke Poech, AI506, Spring 2026

Keywords

Loss functions

- Information Theory
- Maximum Likelihood Estimation
- Binary/Categorical Cross-entropy
- KL divergence
- Mean squared error

Deep Learning Intuition

- Shortcut Learning & Simplicity Bias
- Lottery Ticket Hypothesis
- Residual connections as ensembles of simple functions
- Deep Double Descent & Grokking

Loss functions (continued)

Where we left of: Likelihood

The likelihood of a set of parameters is the probability of observing the data under the model.

Single example: $l(x, \theta) = p(x|\theta)$

Entire dataset: $L(x, \theta) = \prod_{x \in \mathcal{X}} p(x|\theta)$

In log space: $\log L(X, \theta) = \sum_{x \in \mathcal{X}} \log p(x|\theta)$

Maximum likelihood estimator

$$\theta_{\mathrm{MLE}} = \arg\max_\theta L(X, \theta) =$$
$$\arg\max_\theta \log L(X, \theta)$$

We have seen that we can calculate the argmax analytically for a Gaussian.

Generative vs. discriminative schools

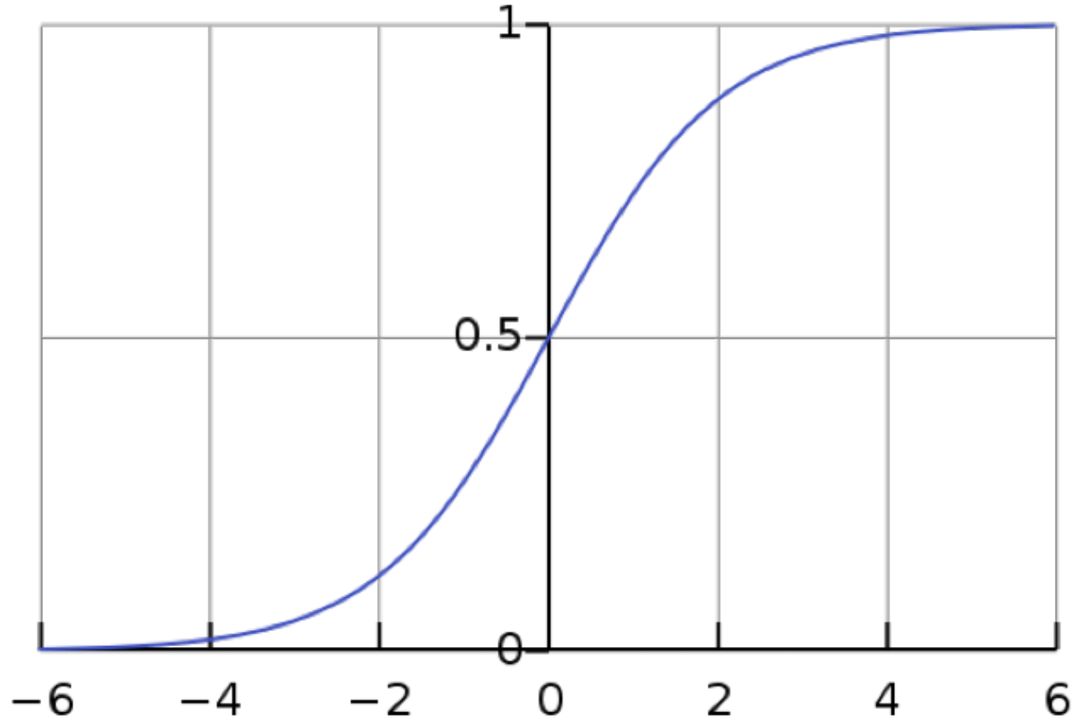
Generative: We must model $p(c, x)$, then we can derive $p(c|x) = \frac{p(x|c) \cdot p(c)}{p(x)}$

Discriminative: Let's estimate $p(c|x)$ directly from an arbitrary input vector!

Discriminative models

$$p(y|x) = \sigma(m(x))$$

with σ the logistic sigmoid function $\sigma(x) = \frac{e^x}{1+e^x}$



Why the logistic sigmoid function?

Maps $(-\infty, +\infty)$ to $(0, 1)$.

Smooth function, simple derivative.

It is mathematically derived from the log odds.

$$\log \text{odds} = \log \frac{p(x)}{1-p(x)}$$

$\sigma(x) = \frac{e^x}{1+e^x}$ and substitute in the log odds for x:

$$\sigma(x) = \frac{\frac{p(x)}{1-p(x)}}{1+\frac{p(x)}{1-p(x)}} = \frac{\frac{p(x)}{1-p(x)}}{\frac{(1-p(x))+p(x)}{1-p(x)}} = \frac{p(x)}{(1-p(x))+p(x)} = p(x)$$

Intermezzo: Information Theory

Shannon's information theory (1948)

Desiderata

- Continuity
- Symmetry
- Maximal value
- Additive

Surprisal

Given a distribution $P(X)$, how can we characterize how surprised we are when we observe a particular event x?

Intution: the smaller the probability of the event, the larger the surprise.

$$\text{Surprisal}(x) = \frac{1}{p(x)}$$

Shannon information as surprisal

$$h(x) = \log \frac{1}{p(x)} = -\log p(x)$$

Key advantage: Additive. Now, you can just add the surprisal of two events.

If $\log_2$ is used, the unit is called *bits*

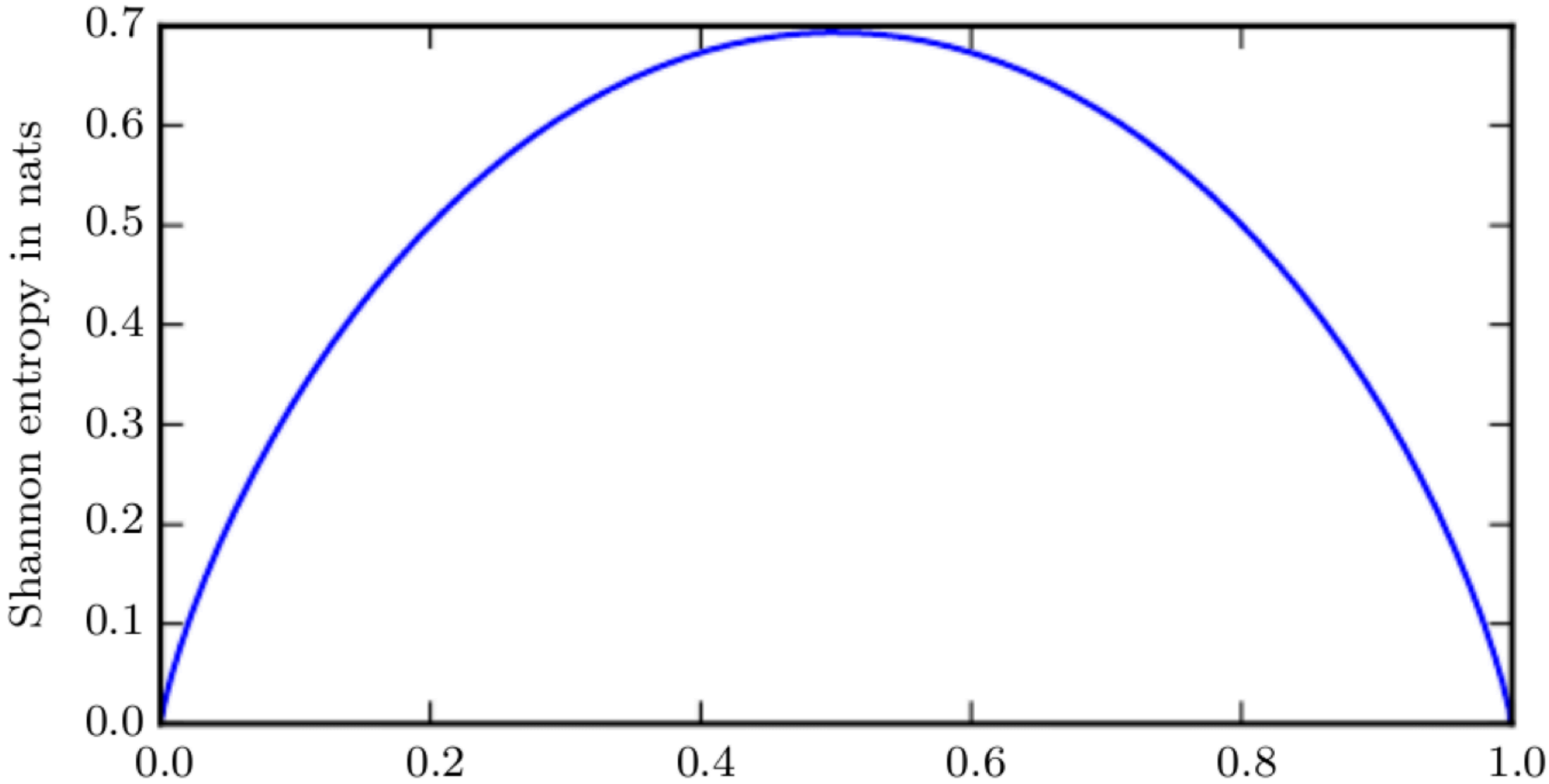
If base e is used, the unit is called *nats*

Entropy = Average Shannon Information

$$H(X) = \mathbb{E}[h(x)] = -\sum p(x) \log p(x)$$

Or, *uncertainty of a random variable.*

Entropy of a biased coin with probability p



Interpretation of Entropy

A variable with an entropy of $H(X)$ bits provides enough Shannon information to choose between $2^{H(X)}$ equally probable alternatives.

Cross-entropy

How surprised are you on average, if you assume a distribution Q, when it's actually P.

$$H(P, Q) = \mathbb{E}_p[h(x)] = -\sum p(x) \log q(x)$$

Back to our loss functions

Let's start with logistic regression

In logistic regression, m is just a linear model

$$m(x) = \theta^T x = \theta_0 + \theta_1 x$$

Then $\sigma(m(x)) = \frac{e^{\theta_0 + \theta_1 x}}{1 + e^{\theta_0 + \theta_1 x}}$

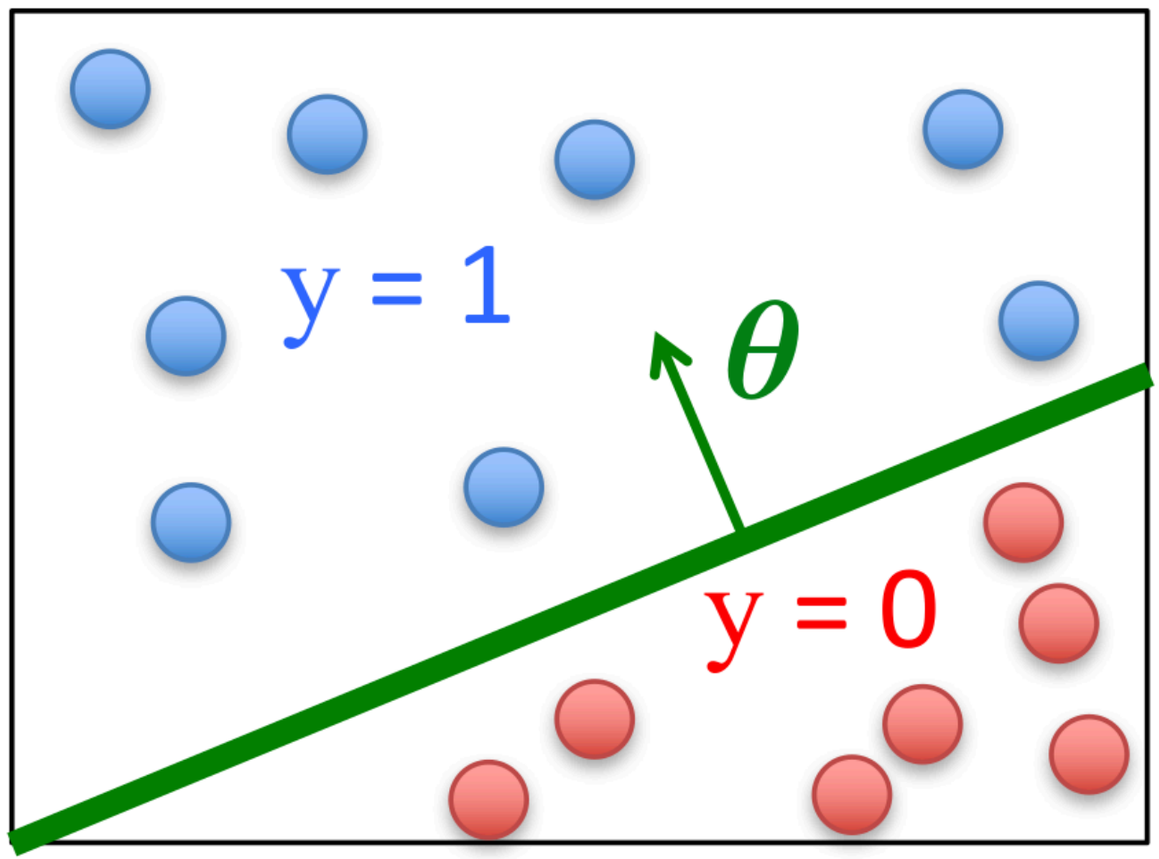
Learning goal

Our model: $p(y|x, \theta) = \sigma(m_\theta(x))$

$m(x)$ should be very negative for negative instances

$m(x)$ should be very positive for positive instances

Select a threshold on $\sigma(m(x))$ to make the decision. Often 0.5.



With loss function is suitable for classification?

Mean squared error is not suitable (already non-convex with a simple logistic regression).

Can we get a maximum likelihood estimate?

Binary Cross-Entropy Loss

$$\log L(\theta) = \sum_i y_i \log \sigma(m(x_i)) + (1 - y_i) \log(1 - \sigma(m(x_i)))$$

Maximizing likelihood is equivalent to minimizing cross entropy:

$$\mathcal{L}_{CE}(\theta) = -\log L(\theta) = -\sum_i [y_i \log \sigma(m(x_i)) + (1 - y_i) \log(1 - \sigma(m(x_i)))]$$

where $y_i \in 0, 1$ gives our estimate for $p(x)$ and $q(x)$ comes from the model.

Relationship between KL Divergence and Cross Entropy

$$D_{KL}(P\|Q) = H(P,Q) - H(P)$$

$$D_{KL}(P\|Q) = \sum_i p(x_i) \log \frac{p(x_i)}{q(x_i)} = -\sum_i p(x_i) \log q(x_i) + \sum_i p(x_i) \log p(x_i)$$

Key message: minimizing cross-entropy over model parameters is the same as minimizing KL divergence, since $H(P)$ is constant with respect to θ.

In practice: X-ent is easier to optimize for than KL-div.

Softmax loss

For multi-class classification with K classes, assume targets follow a Categorical distribution

The model outputs a probability vector via softmax:

$$\sigma(m(x))_k = \frac{e^{m(x)_k}}{\sum_{j=1}^{K} e^{m(x)_j}}$$

The likelihood of a single observation with one-hot label $y_i \in \{0,1\}^K$

$$p(y_i|x_i,\theta) = \prod_{k=1}^{K} \sigma(m(x_i))_k^{y_{ik}}$$

Log-likelihood over all observations:

$$\log L(\theta) = \sum_i \sum_k y_{ik} \log \sigma(m(x_i))_k$$

Negating gives cross-entropy loss:

$$\mathcal{L}_{CE}(\theta) = -\sum_i \sum_k y_{ik} \log \sigma(m(x_i))_k$$

Mean-squared error loss

Assume targets are Gaussian: $y_i \sim \mathcal{N}(m(x_i), \sigma^2)$

$$p(y_i|x_i, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - m(x_i))^2}{2\sigma^2}\right)$$

Log likelihood over all observations:

$$\log L(\theta) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_i (y_i - m(x_i))^2$$

Maximizing log likelihood is equivalent to minimizing

$$\mathcal{L}_{MSE} = \sum_i (y_i - m(x_i))^2$$

(because constants can be ignored for minimum)

Keywords

Loss functions

- Information Theory
- Maximum Likelihood Estimation
- Binary/Categorical Cross-entropy
- KL divergence
- Mean squared error

Deep Learning Intuition

- Shortcut Learning & Simplicity Bias
- Lottery Ticket Hypothesis
- Residual connections as ensembles of simple functions
- Deep Double Descent & Grokking

Deep Learning intuition

Shortcut Learning in Deep Neural Networks[1]

1. Geirhos et al., Nature Machine Intelligence 2020 ↩

Shortcut Learning Implications

Deep neural networks tend to find the simplest possible solution, even if it is a shortcut.

Conversely, overparametrization exerts a **simplicity bias**.

The Lottery Ticket Hypothesis[1]

Deep neural networks contain small and sparse subnetworks, that could be trained in isolation to ~same performance as the full network

Depth and overparametrization gives you exponentially many rolls (=tickets), such that it is more likely to find a good subnetwork + initial weights (= a winning ticket).

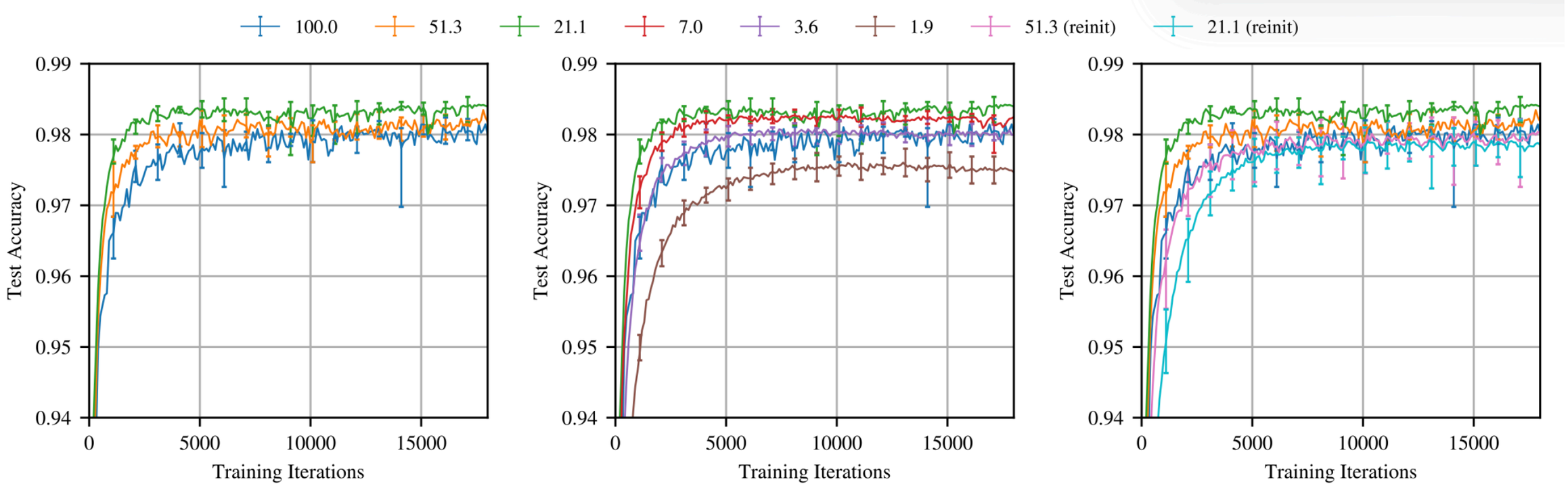
1. Frankle & Carbin, 2019 ICLR ↩

The Lottery Ticket Hypothesis (cont'd)

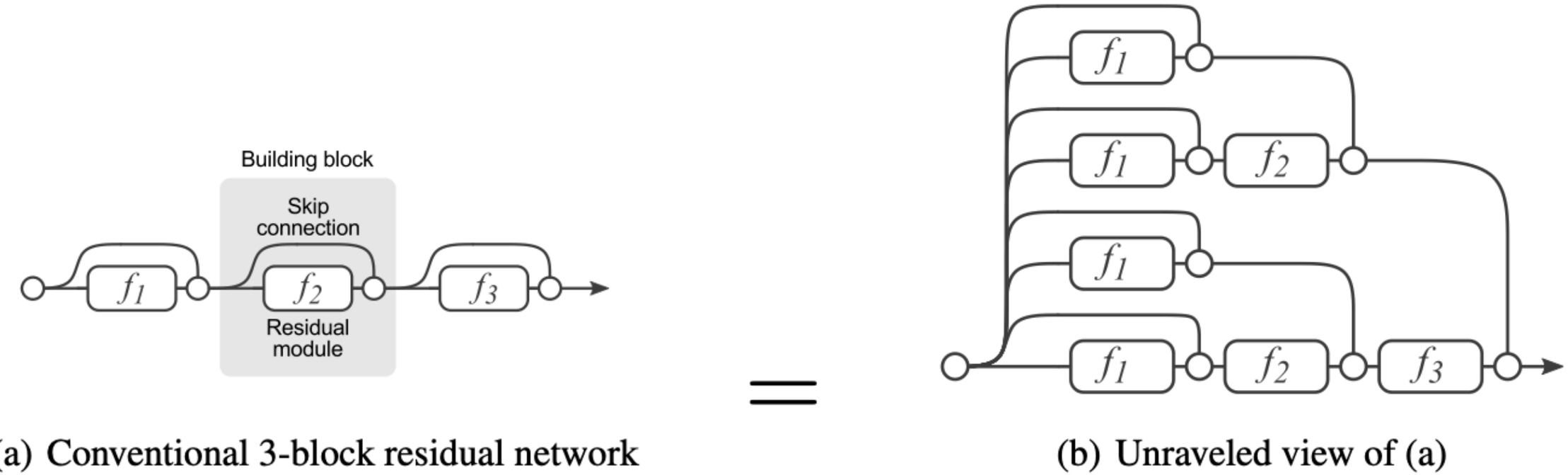
Algorithm for finding winning tickets:

1. Train til convergence.
2. Prune the lowest magnitude weights.
3. Restart with subnetwork@init from Step 1.

The Lottery Ticket Hypothesis (cont'd)



Residual Networks Behave Like Ensembles of Relatively Shallow Networks[1]

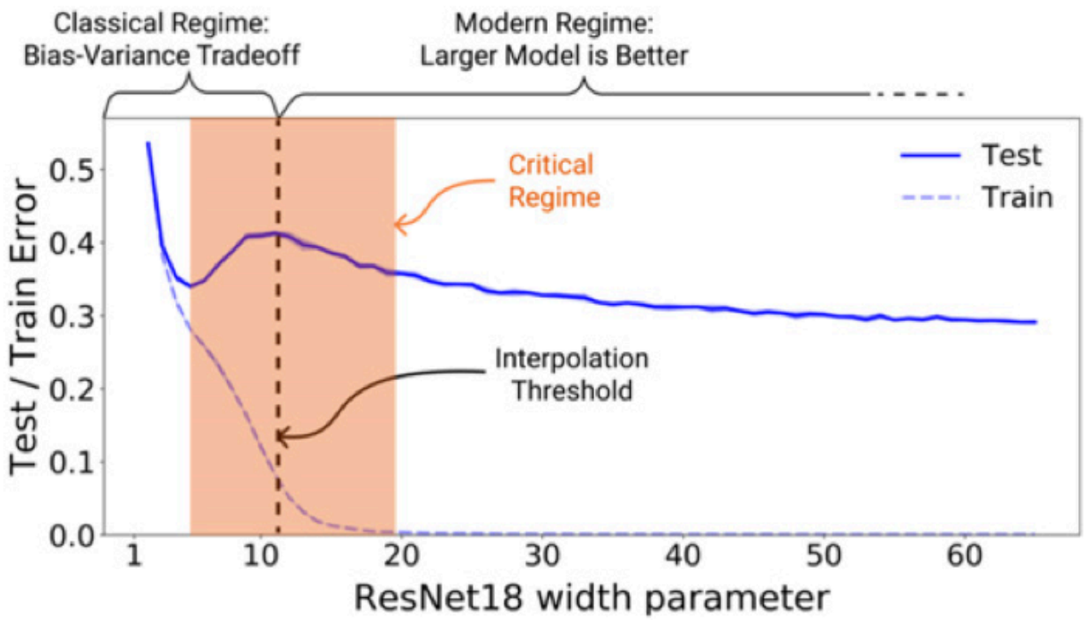


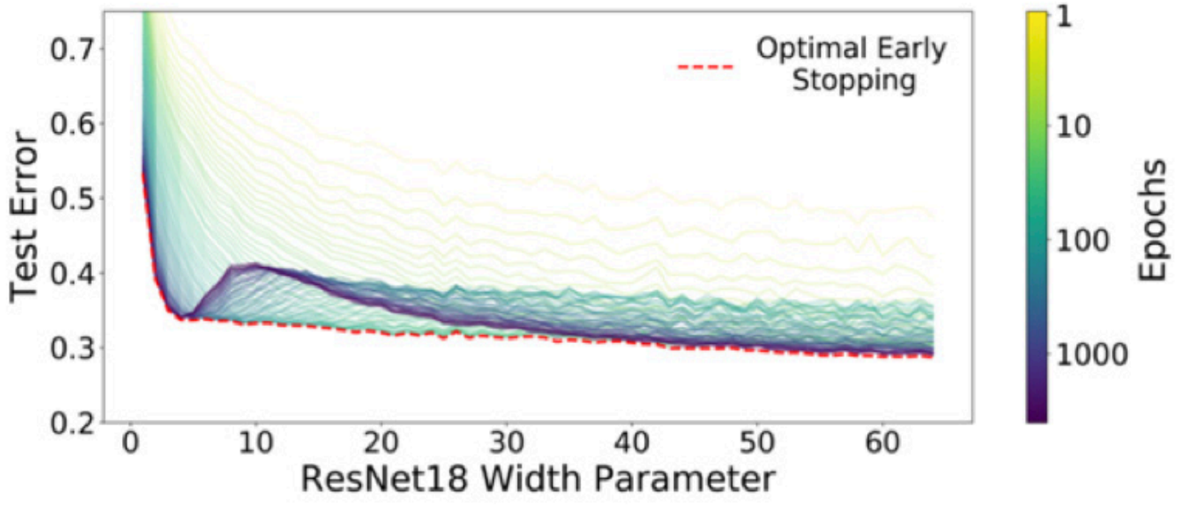
(a) Conventional 3-block residual network

(b) Unraveled view of (a)

1. Veit, Vilber, & Belongie (2016 NeurIPS ↩

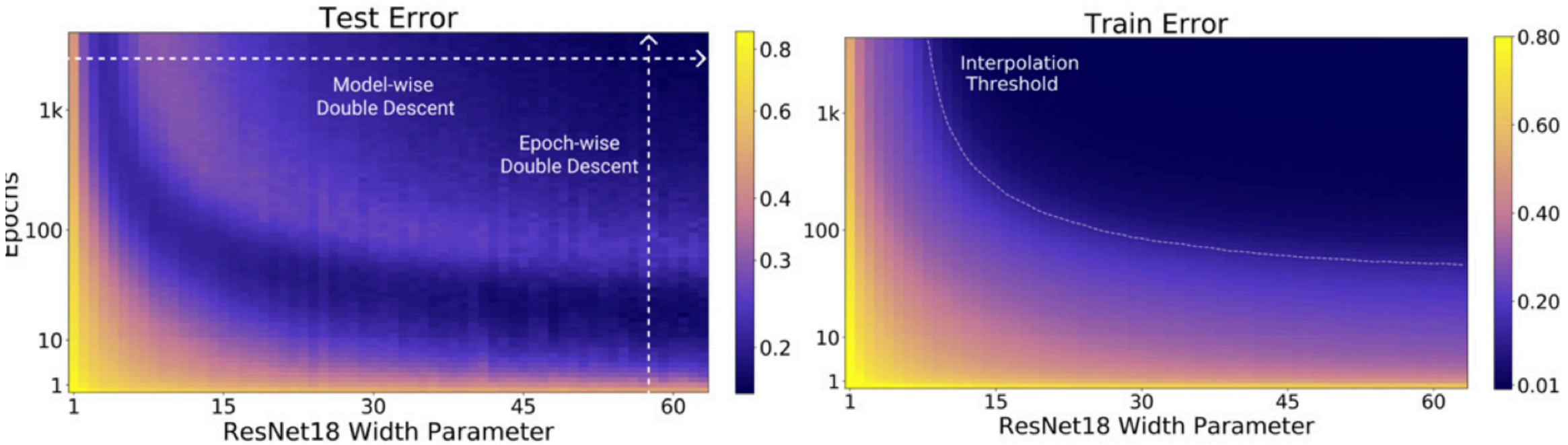
Deep Double Descent[1]





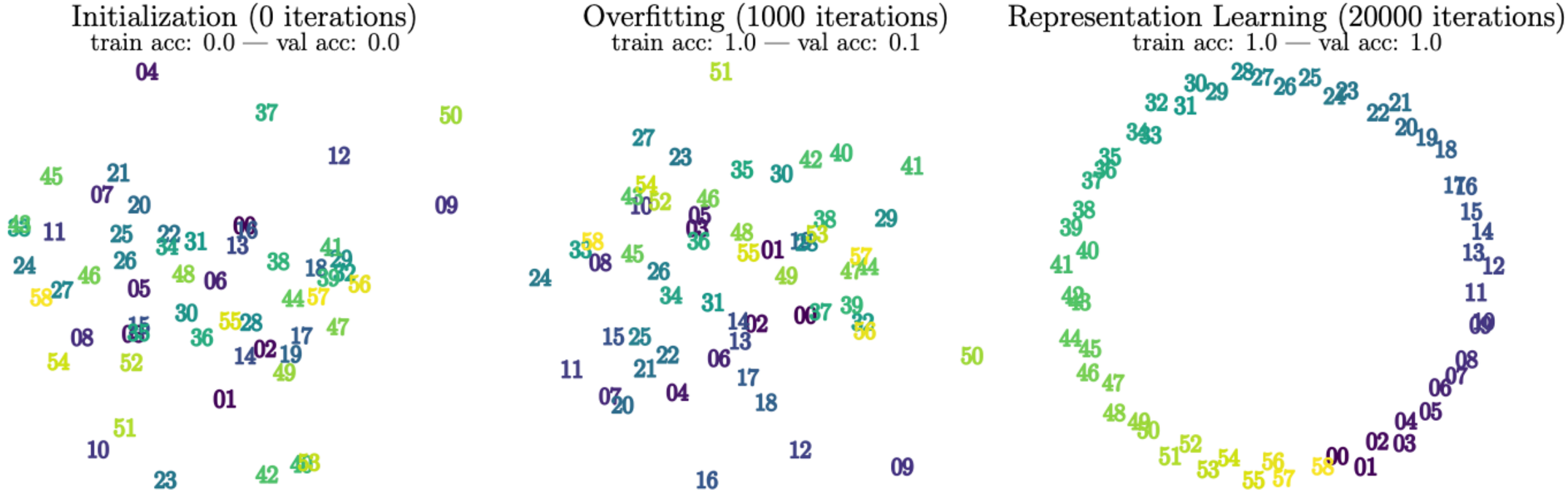
1. Nakkiran et al. (2021) ↩

Deep Double Descent from a different perspective



Grokking[1]

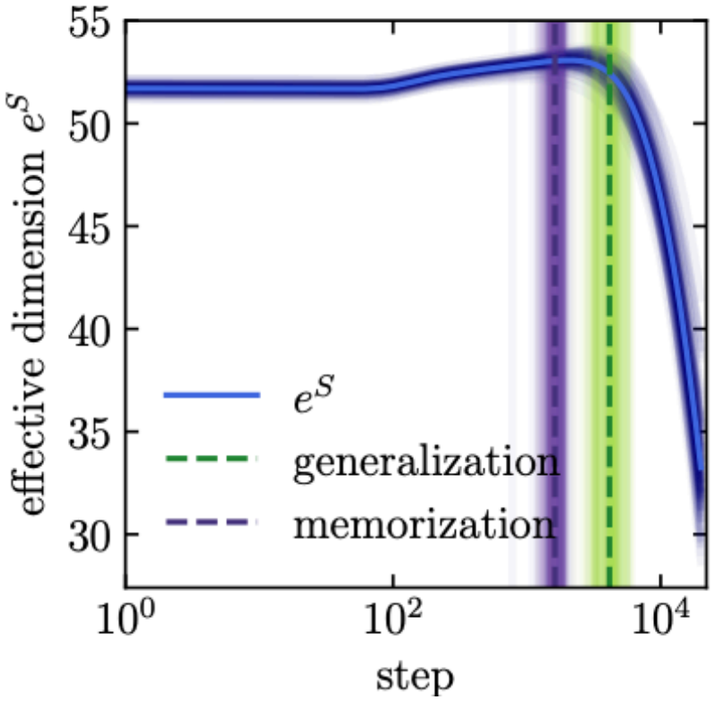
Grokking: a model first memorizes the training data, then — long after — suddenly learns to generalize

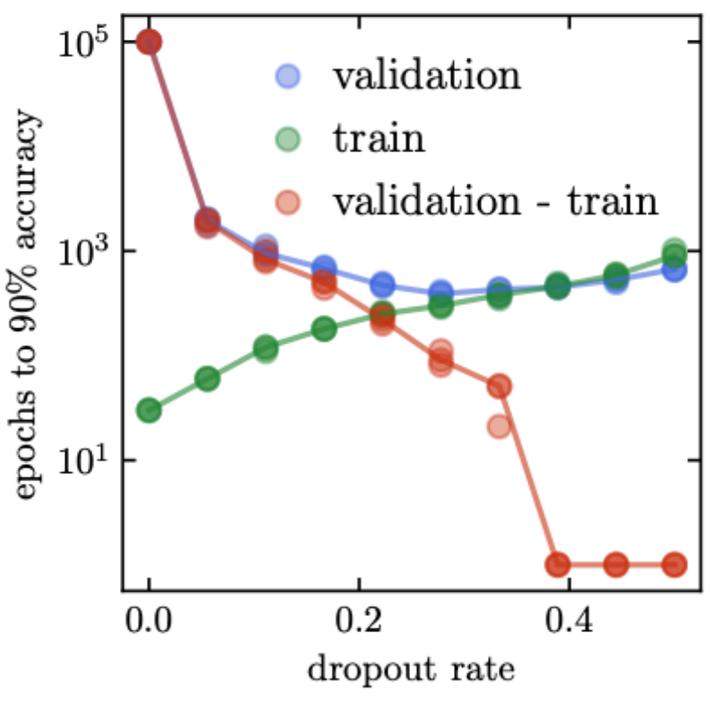


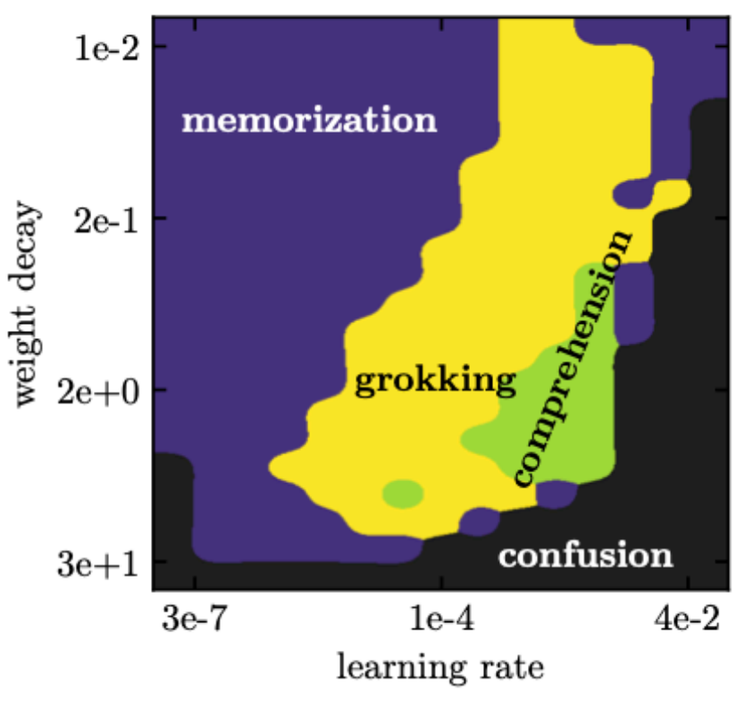
1. Liu et al. (2022 NeurIPS) ↩

More Grokking

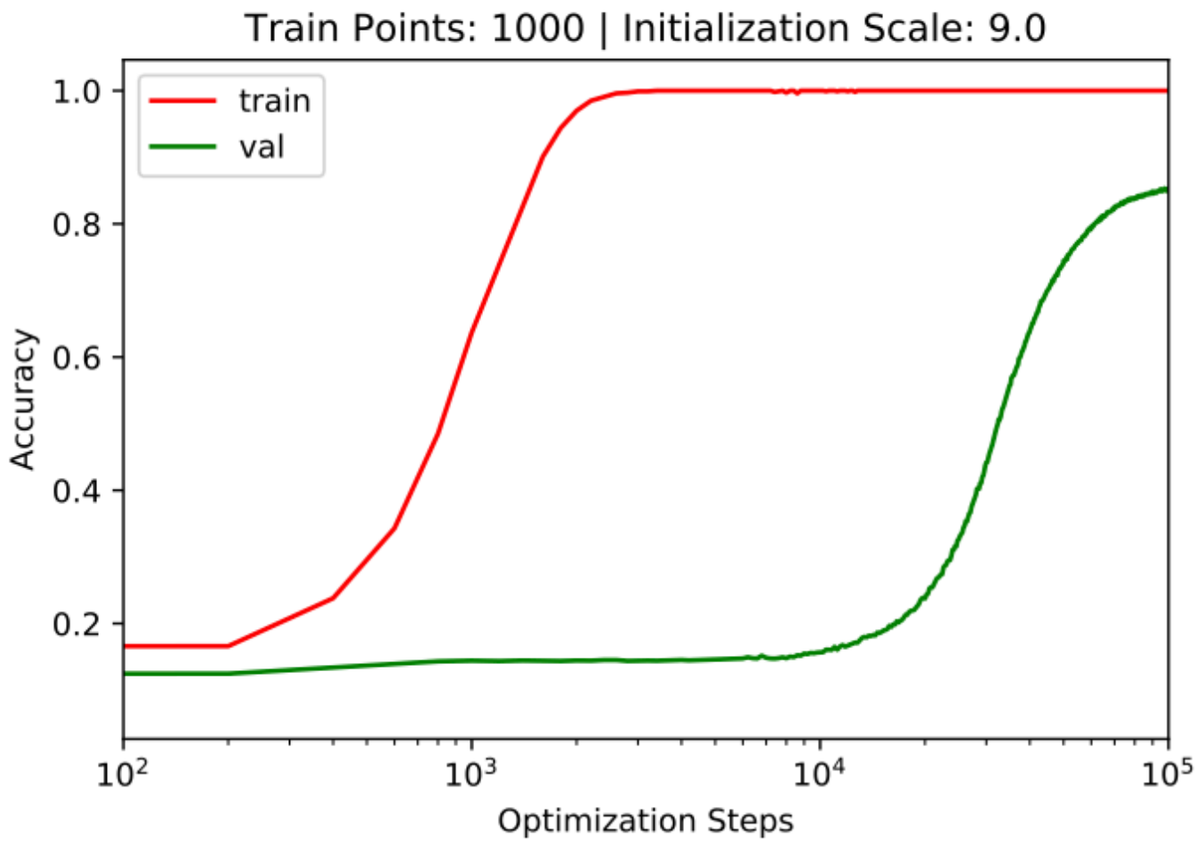
Generalization can happen after overfitting.



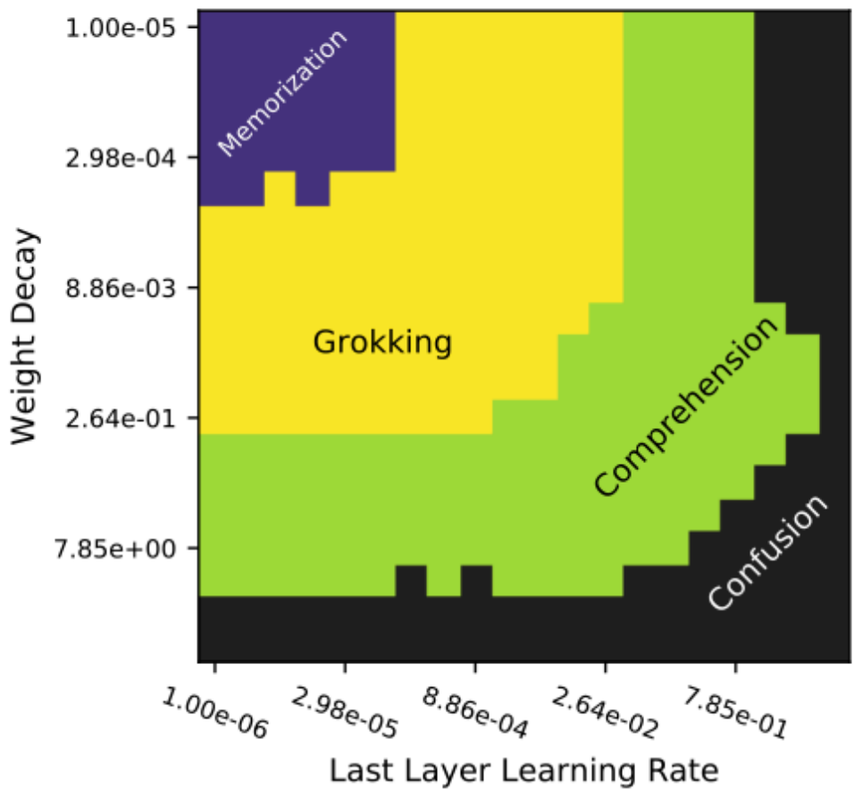




Even More Grokking 2



(a)



(b)

Keywords

Loss functions

- Information Theory
- Maximum Likelihood Estimation
- Binary/Categorical Cross-entropy
- KL divergence
- Mean squared error

Deep Learning Intuition

- Shortcut Learning & Simplicity Bias
- Lottery Ticket Hypothesis
- Residual connections as ensembles of simple functions
- Deep Double Descent & Grokking

How's my teaching?